

SUMMARY RANGES

```
class Solution:
```

```
    def summaryRanges(self, nums):
```

```
        result = []
```

```
        i = 0
```

```
        while i < len(nums):
```

```
            start = nums[i]
```

```
            while i + 1 < len(nums) and nums[i + 1] == nums[i] + 1:
```

```
                i += 1
```

```
            if start == nums[i]:
```

```
                result.append(str(start))
```

```
            else:
```

```
                result.append(str(start) + "->" + str(nums[i]))
```

```
            i += 1
```

```
        return result
```

POWER OF TWO

```
class Solution(object):
```

```
    def isPowerOfTwo(self, n):
```

```
        """
```

```
        :type n: int
```

```
        :rtype: bool
```

```
        """
```

```
        return n > 0 and (n & (n - 1)) == 0
```

IMPLEMENT QUEUE USING STACKS

```
class MyQueue:
```

```
    def __init__(self):
```

```
        self.stack1 = []
```

```
        self.stack2 = []
```

```
    def push(self, x):
```

```
        self.stack1.append(x)
```

```
    def pop(self):
```

```
        self.move()
```

```
        return self.stack2.pop()
```

```
    def peek(self):
```

```
        self.move()
```

```
        return self.stack2[-1]
```

```
    def empty(self):
```

```
        return len(self.stack1) == 0 and len(self.stack2) == 0
```

```
    def move(self):
```

```
        if len(self.stack2) == 0:
```

```
            while len(self.stack1) > 0:
```

```
                self.stack2.append(self.stack1.pop())
```

PALINDROME LINKED LIST

class Solution:

def isPalindrome(self, head):

slow, fast, prev = head, head, None

while fast and fast.next:

slow, fast = slow.next, fast.next.next

prev, slow, prev.next = slow, slow.next, None

while slow:

slow.next, prev, slow = prev, slow, slow.next

fast, slow = head, prev

while slow:

if fast.val != slow.val: return False

fast, slow = fast.next, slow.next

return True

VALID ANAGRAM

```
class Solution:
```

```
    def isAnagram(self, s, t):
```

```
        if len(s) != len(t):
```

```
            return False
```

```
        count = {}
```

```
        for ch in s:
```

```
            count[ch] = count.get(ch, 0) + 1
```

```
        for ch in t:
```

```
            if ch not in count:
```

```
                return False
```

```
            count[ch] -= 1
```

```
            if count[ch] < 0:
```

```
                return False
```

```
        return True
```

BINARY TREE PATHS

```
class Solution:
```

```
    def binaryTreePaths(self, root):
```

```
        result = []
```

```
        def dfs(node, path):
```

```
            if node is None:
```

```
                return
```

```
            path += str(node.val)
```

```
            if node.left is None and node.right is None:
```

```
                result.append(path)
```

```
                return
```

```
            path += "->"
```

```
            dfs(node.left, path)
```

```
            dfs(node.right, path)
```

```
        dfs(root, "")
```

```
        return result
```

ADD DIGITS

```
class Solution:
```

```
    def addDigits(self, num):
```

```
        while num >= 10:
```

```
            total = 0
```

```
            while num > 0:
```

```
                total += num % 10
```

```
                num //= 10
```

```
            num = total
```

```
        return num
```

UGLY NUMBER

```
class Solution:
```

```
    def isUgly(self, n):
```

```
        if n <= 0:
```

```
            return False
```

```
        for p in [2, 3, 5]:
```

```
            while n % p == 0:
```

```
                n //= p
```

```
        return n == 1
```


MISSING NUMBER

class Solution:

def missingNumber(self, nums):

n = len(nums)

expected = n * (n + 1) // 2

actual = sum(nums)

return expected - actual

FIRST BAD VESRSION

```
class Solution:
```

```
    def firstBadVersion(self, n):
```

```
        left = 1
```

```
        right = n
```

```
        while left < right:
```

```
            mid = left + (right - left) // 2
```

```
            if isBadVersion(mid):
```

```
                right = mid
```

```
            else:
```

```
                left = mid + 1
```

```
        return left
```